

Andrew Gard - equitable.equations@gmail.com

Cross validation



Cross validation is a method for estimating the future performance of a model without relying on a separate holdout set. It's often used in machine learning for the tuning of hyperparameters.



To perform cross validation, we first randomly partition the training data into v subsets of approximately equal size. These subsets are called **folds**. While any value of $v \leq n$ is possible, $v = 10$ is a standard default.

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

We then train the model v times. Each time, we omit one of the folds from the training set and instead use it for evaluation. Finally, we average the results to obtain an estimate of the performance of a model trained on the whole set.

When randomly splitting a data set, it's possible that some folds won't be representative of the whole set. For this reason, it's common practice to **stratify** the cross-validation split, ensuring that each fold includes approximately the same distribution of the response variable.



Example. Suppose we want to build a classification algorithm using the `iris` data set, which includes 150 observations of 3 different species.

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

For this example, we'll use a simple KNN model with 11 neighbors and estimate performance using 10-fold cross-validation.

When using $v = 10$ folds, a set with 150 rows has 15 observations per fold. To perform cross-validation, we train the KNN model 10 times on 10 different subsets of 135 observations, evaluating the performance of each using the remaining 15.

I did this using the `tidymodels` framework in R.



The results for fold #1 are shown below. The 15 rows listed have been used as an assessment set for a model built with the other 135.

id	.row	species	.pred_class
Fold01	7	setosa	setosa
Fold01	32	setosa	setosa
Fold01	33	setosa	setosa
Fold01	40	setosa	setosa
Fold01	43	setosa	setosa

id	.row	species	.pred_class
Fold01	52	versicolor	versicolor
Fold01	54	versicolor	versicolor
Fold01	57	versicolor	versicolor
Fold01	70	versicolor	versicolor
Fold01	84	versicolor	virginica

id	.row	species	.pred_class
Fold01	106	virginica	virginica
Fold01	110	virginica	virginica
Fold01	122	virginica	virginica
Fold01	124	virginica	virginica
Fold01	130	virginica	virginica

Overall, this model correctly classified 14/15 observations, an accuracy rate of about 93.3%.

Each of the 10 folds will result in an accuracy metric like this one. Averaging these gives a reasonable estimate of how a KNN-model trained on the whole set will perform.

$$\begin{aligned}\text{CV accuracy} &= \frac{1}{v} \sum_{i=1}^v (\text{Accuracy evaluated on } i^{th} \text{ fold}) \\ &\approx \text{Prediction accuracy}\end{aligned}$$

Using the same procedure 10 times yields the following results.

id	.metric	.estimate
Fold01	accuracy	0.9333333
Fold02	accuracy	0.8666667
Fold03	accuracy	0.9333333
Fold04	accuracy	1.0000000
Fold05	accuracy	1.0000000

id	.metric	.estimate
Fold06	accuracy	1.0000000
Fold07	accuracy	0.9333333
Fold08	accuracy	0.9333333
Fold09	accuracy	0.9333333
Fold10	accuracy	0.9333333

Overall, $142/150 \approx 94.7\%$ of observations have been properly classified.



Because each of these 10 models is trained on only 90% of the data, CV accuracy is generally slightly lower than it would be for a model trained on the entire training set.

Using more folds would reduce this bias, though the computational cost would be higher.





Cross-validation is particularly useful when tuning a model's hyperparameters.

A **hyperparameter** (or **tuning parameter**) is a coefficient chosen by the user which controls some aspect of the model's behavior, often related to its overall complexity or tightness of fit to the training set.



A few examples of hyperparameters:

- The number of neighbors used in a KNN-model
- The coefficient of shrinkage in ridge or lasso regression
- The complexity cost of nodes in a decision tree

In each case, there is no single value guaranteed to be theoretically best. Typically, we fit the model repeatedly using different values of the hyperparameter, then use cross-validation to decide empirically which value is most appropriate.



In our iris example, the choice to use 11 neighbors was more or less arbitrary. To tune this hyperparameter using cross validation, we would build a separate KNN model on each fold for a range of k 's (the so-called **search grid**), compute the same statistics as before, and compare performance.



Results of this tuning are shown below.

k	Accuracy	SE
9	0.9466667	0.01333333
11	0.9466667	0.01333333
13	0.9466667	0.01333333
15	0.9533333	0.01018350
17	0.9600000	0.01088662
19	0.9600000	0.01088662
21	0.9600000	0.01088662

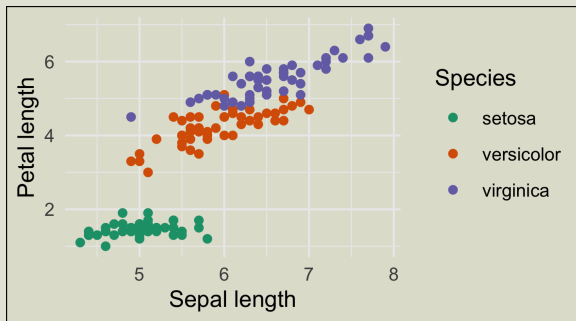
k	Accuracy	SE
23	0.9600000	0.01088662
25	0.9533333	0.01018350
27	0.9533333	0.01018350
29	0.9533333	0.01018350
31	0.9533333	0.01018350
33	0.9533333	0.01018350
35	0.9533333	0.01018350

k	Accuracy	SE
37	0.9533333	0.010183502
39	0.9466667	0.008888889
41	0.9533333	0.010183502
43	0.9533333	0.010183502
45	0.9533333	0.010183502
47	0.9466667	0.013333333
49	0.9466667	0.013333333

k	Accuracy	SE
51	0.9533333	0.01422916
53	0.9466667	0.01662959
55	0.9400000	0.02096440
57	0.9400000	0.02096440
59	0.9400000	0.02096440
61	0.9333333	0.02434322
63	0.9333333	0.02434322

Accuracy is slightly higher around $k = 21$, though many results are clustered within one SE of that maximum accuracy rate.

It's not hard to see why the knn-model is relatively indifferent to the number of neighbors used.



At the end of a tuning procedure, a single value is chosen for the hyperparameter and a final model is constructed using the full training set. The most obvious choice is the value that optimizes the metric being used (accuracy in our example), but when multiple values are competitive with one another, practitioners often prefer the one corresponding to the less flexible model.



The 1-SE rule recommends adopting the least flexible model whose cross-validation performance (accuracy in this case) lies within one standard error of the optimizing value.



In our iris example, the maximum accuracy is .960, in which case the standard error is .011.

k	Accuracy	SE
9	0.9466667	0.01333333
11	0.9466667	0.01333333
13	0.9466667	0.01333333
15	0.9533333	0.01018350
17	0.9600000	0.01088662
19	0.9600000	0.01088662
21	0.9600000	0.01088662

k	Accuracy	SE
23	0.9600000	0.01088662
25	0.9533333	0.01018350
27	0.9533333	0.01018350
29	0.9533333	0.01018350
31	0.9533333	0.01018350
33	0.9533333	0.01018350
35	0.9533333	0.01018350

k	Accuracy	SE
37	0.9533333	0.010183502
39	0.9466667	0.008888889
41	0.9533333	0.010183502
43	0.9533333	0.010183502
45	0.9533333	0.010183502
47	0.9466667	0.013333333
49	0.9466667	0.013333333

k	Accuracy	SE
51	0.9533333	0.01422916
53	0.9466667	0.01662959
55	0.9400000	0.02096440
57	0.9400000	0.02096440
59	0.9400000	0.02096440
61	0.9333333	0.02434322
63	0.9333333	0.02434322

We choose the least flexible model with CV accuracy better than .949.
Using the 1-SE rule, we might adopt $k = 51$.

Why not just use a holdout set to compare competing models? At least two major reasons:

- Cross-validation averages results over many holdout sets, yielding more reliable performance estimates.
- If a holdout set is used to compare hyperparameter values, it no longer provides an unbiased estimate of the performance of the selected model.



On the other hand, cross-validation can be computationally expensive, fitting each candidate model once per fold.

For instance, to search just 10 candidate values for each of 2 hyperparameters using 10-fold cross-validation requires 1000 fits, plus one more to train the final model on the entire set.



Cross-validation is a standard part of a machine learning workflow.

Typical Machine Learning Workflow

